



Universidad Alfonso X El Sabio

Grado en Ingeniería Matemática
Simulación Numérica

INFORME 1: EXAMEN PARCIAL

AUTOR:

González García, María
López Rodríguez, Claudia

3 de marzo de 2025

Resumen

En el presente informe se lleva a cabo un análisis detallado de un conjunto de datos experimentales mediante la aplicación de diversos métodos numéricos de interpolación y ajuste. Se implementan polinomios de interpolación de Lagrange y Splines Cúbicos, así como ajustes por regresión lineal y no lineal, considerando modelos polinómicos de distintos grados y funciones exponenciales. Para ello, se desarrollan algoritmos en Python que permiten realizar estos cálculos de manera eficiente y precisa. Finalmente, se presentan y discuten los resultados obtenidos, evaluando la precisión y aplicabilidad de cada método en el contexto de la simulación numérica.

Índice

1. Introducción.

El examen consistirá en hacer un análisis de datos experimentales. Cada grupo tendrá que elegir uno de los 6 archivos que se encuentran en la plataforma, denominados **Datos Grupos N.txt**, donde N representa el número del grupo.

Con los datos dados en el archivo, los integrantes tendrán que realizar los siguientes ajustes:

1. Desarrollar una función en Python que determine el polinomio de interpolación de Lagrange que se ajuste a los datos entregados.
2. Desarrollar una función en Python que determine los polinomios de interpolación mediante Splines Cúbicos.
3. Ajustar los datos entregados mediante regresión lineal a polinomios de grado 1, 2, 3 y 4, y a la función $y = a_0x^{a_1}$ haciendo una transformación de los datos.
4. Ajustar los datos entregados mediante regresión no lineal a las siguientes funciones:

a) $f(x) = a_0 - a_1e^{-a_2x}$

b) $f(x) = a_0x - a_1e^{-a_2x}$

c) $f(x) = a_0x^2 - a_1e^{-a_2x}$

En virtud de que nuestro estudio se enmarca dentro del Grupo 2, se procederá a emplear el archivo: **Datos Grupo 2.txt**. Seguidamente, se presenta una representación gráfica de los datos originales (véase la Figura ??).

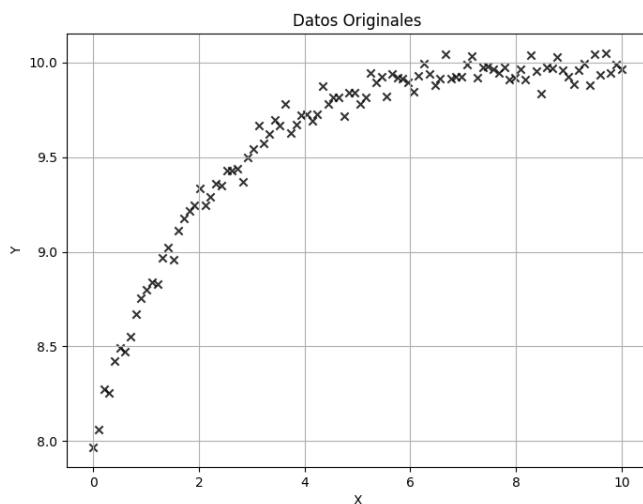


Figura 1: Distribución de los datos del archivo.

2. Métodos y modelos matemáticos.

2.1. Interpolación.

2.1.1. Interpolación de Newton.

La **interpolación de Newton** es un método numérico utilizado para encontrar un polinomio que pase exactamente por un conjunto de puntos dados. Este método se basa en la construcción de un **polinomio de diferencias divididas**, lo que permite una formulación fácil de actualizar cuando se agregan nuevos puntos.

Definición del Polinomio de Newton. Dado un conjunto de $n+1$ puntos distintos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, el polinomio interpolador de Newton se define como:

$$f_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots + f[x_0, x_1, \dots, x_n] \prod_{i=0}^{n-1} (x - x_i). \quad (1)$$

Diferencias divididas. Los coeficientes del polinomio se calculan mediante **diferencias divididas**, las cuales se definen recursivamente de la siguiente manera:

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}. \quad (2)$$

Este enfoque permite que el cálculo del polinomio sea altamente eficiente, ya que cada diferencia dividida de orden superior se construye a partir de diferencias divididas de orden inferior, lo que reduce significativamente el costo computacional. La Figura ?? ilustra la estructura recursiva del cálculo de diferencias divididas, donde cada término de orden superior depende de los términos de orden inferior.

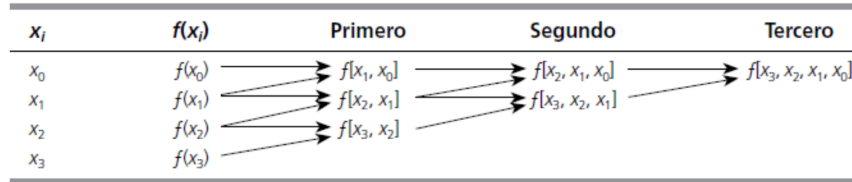


Figura 2: Representación gráfica de la naturaleza recursiva de las diferencias divididas finitas.

Dado que la construcción del polinomio de Newton es incremental, es posible construir $f_n(x)$ a partir de $f_{n-1}(x)$, lo que hace que este método sea eficiente para cálculos iterativos. Esto es particularmente útil cuando se agregan nuevos datos al conjunto original, ya que no es necesario recalcular completamente el polinomio, sino únicamente actualizar los términos adicionales.

Error de la Interpolación de Newton. El error de interpolación de Newton nos permite estimar qué tan precisa es la aproximación del polinomio interpolador respecto a la función original. La fórmula del error se expresa como:

$$R_n(x) = f[x_0, x_1, \dots, x_n, x] \prod_{i=0}^n (x - x_i). \quad (3)$$

Si asumimos que la función $f(x)$ es $n + 1$ veces derivable en un intervalo que contiene los puntos $x_0, x_1, \dots, x_n$, el término de error puede expresarse en términos de la derivada de orden $n + 1$:

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i), \quad \text{dónde } \xi \in [x_0, x_n]. \quad (4)$$

2.1.2. Interpolación de Lagrange.

La **interpolación de Lagrange** es un método numérico utilizado para encontrar un polinomio de grado n que pasa exactamente por un conjunto de $n+1$ puntos dados $[?, ?]$. Este método se basa en la construcción de una combinación lineal de **polinomios base de Lagrange**, evitando la necesidad de calcular diferencias divididas, como en la interpolación de Newton.

Definición del Polinomio de Lagrange. Dado un conjunto de $n+1$ puntos distintos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, el polinomio interpolador de Lagrange se define como $[?]$:

$$f_n(x) = \sum_{i=0}^n f(x_i) L_i(x), \quad (5)$$

dónde los **polinomios base de Lagrange** $L_i(x)$ están dados por:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (6)$$

Cada $L_i(x)$ es un polinomio de grado n que toma el valor de 1 en x_i y 0 en los demás puntos del conjunto. Esto garantiza que el polinomio interpolador $f_n(x)$ pase exactamente por los valores $f(x_i)$.

Propiedades del Polinomio de Lagrange. El método de interpolación de Lagrange tiene varias características importantes:

- **Evita el cálculo de diferencias divididas**, lo que facilita su implementación computacional.
- **Los puntos de interpolación pueden estar en cualquier orden** y no necesitan ser equidistantes.
- Es un método **global**, ya que cualquier modificación en un solo punto afecta a todo el polinomio interpolador.
- Para cada nuevo punto agregado, es necesario recalcular todos los polinomios base $L_i(x)$.

Su principal desventaja es su inestabilidad para valores grandes de n , lo que puede generar oscilaciones no deseadas en la función interpolante $[?]$. En casos dónde se requiera mayor estabilidad, se recomienda utilizar métodos alternativos como los **Splines Cúbicos** (véase Sección ??).

2.1.3. Interpolación por Splines Cúbicos

La **interpolación por Splines Cúbicos** es un método numérico utilizado para aproximar una función mediante polinomios de tercer grado (*Splines Cúbicos*) en cada subintervalo del dominio $[?, ?]$. A diferencia de la interpolación polinómica global, los Splines Cúbicos garantizan una mayor estabilidad y suavidad, evitando oscilaciones extremas en los extremos del intervalo.

Definición del Spline Cúbico. Dado un conjunto de $n + 1$ puntos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, se define un conjunto de polinomios cúbicos $S_i(x)$ para cada subintervalo $[x_i, x_{i+1}]$ [?]:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, \quad \text{para } x \in [x_i, x_{i+1}]. \quad (7)$$

Estos polinomios se construyen de manera que se garantice la **continuidad** de la función interpoladora y sus derivadas hasta el orden 2 [?].

Condiciones de continuidad. Para garantizar que la interpolación por Splines Cúbicos sea suave y precisa, se deben cumplir las siguientes condiciones:

- **Condición de interpolación:** Cada spline pasa exactamente por los puntos de datos:

$$S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}.$$

- **Continuidad de la primera derivada:** Las derivadas de primer orden deben ser continuas en los puntos de unión:

$$S'_i(x_{i+1}) = S'_{i+1}(x_{i+1}).$$

- **Continuidad de la segunda derivada:** Las derivadas de segundo orden también deben ser continuas:

$$S''_i(x_{i+1}) = S''_{i+1}(x_{i+1}).$$

- **Condición de contorno:** Para definir un conjunto único de Splines Cúbicos, se requieren condiciones adicionales en los extremos:

$$S''_0(x_0) = 0, \quad S''_{n-1}(x_n) = 0$$

Construcción del sistema de ecuaciones. Para determinar los coeficientes a_i, b_i, c_i, d_i , se debe resolver un sistema de ecuaciones que garantice la continuidad de la primera y segunda derivada en cada subintervalo. La clave en la construcción de los Splines Cúbicos es calcular las **segundas derivadas** $M_i = S''_i(x_i)$ en cada nodo.

Planteamiento del sistema de ecuaciones. Dado un conjunto de $n + 1$ puntos de interpolación $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, el sistema de ecuaciones se expresa como:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right), \quad (8)$$

dónde $h_i = x_{i+1} - x_i$ representa la distancia entre nodos consecutivos.

Este sistema se puede expresar en forma matricial como:

$$\begin{bmatrix} 2(h_0 + h_1) & h_1 & 0 & \dots & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & \dots & 0 \\ 0 & h_2 & 2(h_2 + h_3) & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & h_{n-2} \\ 0 & 0 & 0 & h_{n-2} & 2(h_{n-2} + h_{n-1}) \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ M_3 \\ \vdots \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} 6 \left(\frac{y_2 - y_1}{h_1} - \frac{y_1 - y_0}{h_0} \right) \\ 6 \left(\frac{y_3 - y_2}{h_2} - \frac{y_2 - y_1}{h_1} \right) \\ 6 \left(\frac{y_4 - y_3}{h_3} - \frac{y_3 - y_2}{h_2} \right) \\ \vdots \\ 6 \left(\frac{y_n - y_{n-1}}{h_{n-1}} - \frac{y_{n-1} - y_{n-2}}{h_{n-2}} \right) \end{bmatrix}$$

Resolución del sistema de ecuaciones. Una vez resueltas las segundas derivadas M_i , los coeficientes del Spline Cúbico se obtienen mediante:

$$\begin{cases} a_i = y_i, \\ b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{6}(2M_i + M_{i+1}), \\ c_i = \frac{M_i}{2}, \\ d_i = \frac{M_{i+1} - M_i}{6h_i}. \end{cases}$$

Propiedades de los Splines Cúbicos. Los Splines Cúbicos presentan varias ventajas sobre la interpolación polinómica global [?, ?].

- **Mayor estabilidad numérica:** Al ser polinomios de grado bajo en cada subintervalo, se evitan las oscilaciones extremas presentes en la interpolación de Lagrange o Newton.
- **Mayor precisión:** Se ajustan mejor a funciones con cambios de curvatura sin necesidad de polinomios de grado elevado.
- **Continuidad garantizada:** Las derivadas hasta el segundo orden son continuas, proporcionando una transición suave entre intervalos.

2.2. Regresión por mínimos cuadrados.

Un problema al hacer ajustes de curvas, viene dado cuándo los datos tienen errores sustanciales. De este modo, es posible que una interpolación de datos intermedios no sea apropiada. Una forma de hacerlo es obtener una curva que minimice la discrepancia entre los puntos y la curva, esto se denomina **regresión por mínimos cuadrados**.

2.2.1. Regresión Lineal.

La **regresión lineal** busca modelar la relación entre una variable dependiente y y una variable independiente x mediante una ecuación de primer grado [?, ?].

$$y = a_0 + a_1x + \varepsilon. \quad (9)$$

dónde:

- a_0 es la ordenada al origen.
- a_1 es la pendiente de la recta, que indica cómo varía y con respecto a x .
- ε es el error asociado a cada observación.

El objetivo del modelo es encontrar los valores de a_0 y a_1 que minimicen la **suma de los errores cuadráticos**, definida como:

$$S_r = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - a_0 - a_1x_i)^2.$$

Estimación de los coeficientes. Los coeficientes a_0 y a_1 se determinan mediante el **método de mínimos cuadrados** [?]. Se deriva la suma de los errores cuadráticos con respecto a estos coeficientes y se iguala a cero:

$$S_r = \sum_{i=1}^n (y_i - a_0 - a_1x_i)^2. \quad (10)$$

Derivando con respecto a a_0 y a_1 , obtenemos el siguiente sistema de ecuaciones normales:

$$\begin{cases} na_0 + a_1 \sum x_i = \sum y_i, \\ a_0 \sum x_i + a_1 \sum x_i^2 = \sum x_i y_i. \end{cases} \quad (11)$$

Este sistema de ecuaciones se puede expresar en forma matricial como:

$$\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}.$$

Si despejamos los coeficientes del sistema de ecuaciones normales, obtenemos las siguientes expresiones:

$$\begin{cases} a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \\ a_0 = \bar{y} - a_1 \bar{x} \end{cases}$$

Este sistema garantiza que la recta encontrada minimiza la suma de los errores al cuadrado.

Errores de la Regresión Lineal.

1. Error de la Regresión Lineal.

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2$$

- 2. Error Estándar de Estimación.** La desviación estándar y esta medida miden ambas la dispersión. Sin embargo, la diferencia notoria es que la desviación estándar mide la dispersión de los valores y con respecto a su media (sin considerar x), mientras que el **error estándar** de estimación mide la dispersión de los valores y con respecto a la recta de regresión (considerando x).

$$S_{y/x} = \sqrt{\frac{S_r}{n-2}}$$

Coefficiente de Determinación: r^2 . Para evaluar la calidad del ajuste, se utiliza el **coeficiente de determinación** R^2 , que mide la proporción de variabilidad en y explicada por el modelo [?].

$$r^2 = \frac{S_t - S_r}{S_t} \quad \text{siendo } S_t \text{ la desviación estándar para los datos } y_i. \quad (12)$$

El coeficiente r^2 toma valores entre 0 y 1:

- $r^2 = 1$ indica que el modelo explica el 100% de la variabilidad de y .
- $r^2 = 0$ indica que el modelo no explica nada de la variabilidad de y .

Coefficiente de Correlación: r . Es simplemente la raíz del coeficiente de determinación, cuánto más se aproxime el valor a 1, mejor es el ajuste.

$$r = \sqrt{\frac{S_t - S_r}{S_t}} \quad \text{siendo } S_t \text{ la desviación estándar para los datos } y_i. \quad (13)$$

Limitaciones de la Regresión Lineal. Aunque la regresión lineal es una técnica poderosa, presenta algunas limitaciones:

- **No captura relaciones no lineales** entre las variables.
- Es sensible a **valores atípicos**, los cuales pueden afectar significativamente la estimación de los coeficientes.
- Supone que los errores son independientes y tienen varianza constante.

Cuando estas condiciones no se cumplen, pueden ser necesarios otros modelos como la regresión polinomial (véase Sección ??) o regresión no lineal (véase Sección ??).

2.2.2. Regresión Polinomial.

La **regresión polinomial** es una extensión de la regresión lineal que permite ajustar modelos más flexibles cuando la relación entre las variables no es lineal [?, ?]. En lugar de una línea recta, se ajusta un polinomio de grado n a los datos.

Modelo de regresión polinomial. El modelo matemático para una regresión polinomial de grado n es:

$$y = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n + \varepsilon, \quad (14)$$

dónde:

- y es la variable dependiente, x es la variable independiente y ε es el término de error.
- $a_0, a_1, \dots, a_n$ son los coeficientes del polinomio.

Método de Mínimos Cuadrados. Para determinar los coeficientes $a_0, a_1, \dots, a_n$, se utiliza el **método de mínimos cuadrados**, similar a la regresión lineal [?].

$$\sum_{i=1}^n y_i = a_0 \sum_{i=1}^n 1 + a_1 \sum_{i=1}^n x_i + a_2 \sum_{i=1}^n x_i^2 + \cdots + a_n \sum_{i=1}^n x_i^n.$$

Este sistema se resuelve generalmente mediante álgebra matricial:

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 & \cdots & \sum x_i^n \\ \sum x_i & \sum x_i^2 & \sum x_i^3 & \cdots & \sum x_i^{n+1} \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 & \cdots & \sum x_i^{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_i^n & \sum x_i^{n+1} & \sum x_i^{n+2} & \cdots & \sum x_i^{2n} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \\ \vdots \\ \sum x_i^n y_i \end{bmatrix}$$

Elección del Grado del Polinomio. Uno de los aspectos clave de la regresión polinomial es la elección del grado n . Si n es muy bajo, el modelo puede ser demasiado simple y no capturar correctamente la tendencia de los datos (**subajuste**). Si n es demasiado alto, el modelo puede ajustarse demasiado a los datos de entrenamiento y perder capacidad de generalización (**sobreaajuste**).

En la práctica, el **grado del polinomio se elige analizando el error de estimación** y validando el modelo con datos adicionales [?].

Error estándar de estimación. En el caso de la **regresión polinomial**, el error estándar de estimación $s_{y/x}$ mide la dispersión de los valores observados y_i en torno a la curva ajustada del polinomio. Se calcula como:

$$s_{y/x} = \sqrt{\frac{S_r}{n - (m + 1)}} \quad (15)$$

dónde:

- n es el número total de observaciones en el conjunto de datos.
- m es el **grado del polinomio** ajustado.

2.2.3. Regresión Lineal Múltiple.

La **regresión lineal múltiple** es una extensión de la regresión lineal simple (véase Sección ??) que permite modelar la relación entre una variable dependiente y y múltiples variables independientes ($[?]$) $x_1, x_2, \dots, x_m$. Se utiliza cuando la variable de interés depende de más de un factor.

Modelo de regresión lineal múltiple. El modelo matemático se expresa como:

$$y = a_0 + a_1x_1 + a_2x_2 + \dots + a_mx_m + \varepsilon, \quad (16)$$

Estimación de los coeficientes mediante Mínimos Cuadrados. Al igual que en la regresión lineal simple, los coeficientes $a_0, a_1, \dots, a_m$ se determinan minimizando la suma de los errores cuadráticos. El sistema de ecuaciones normal asociado se expresa en forma matricial como:

$$\begin{bmatrix} n & \sum x_{1i} & \sum x_{2i} & \dots & \sum x_{mi} \\ \sum x_{1i} & \sum x_{1i}^2 & \sum x_{1i}x_{2i} & \dots & \sum x_{1i}x_{mi} \\ \sum x_{2i} & \sum x_{1i}x_{2i} & \sum x_{2i}^2 & \dots & \sum x_{2i}x_{mi} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_{mi} & \sum x_{1i}x_{mi} & \sum x_{2i}x_{mi} & \dots & \sum x_{mi}^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_{1i}y_i \\ \sum x_{2i}y_i \\ \vdots \\ \sum x_{mi}y_i \end{bmatrix}$$

Este sistema se resuelve utilizando álgebra matricial mediante la inversa de la matriz de coeficientes $[?]$.

$$A = (X^T X)^{-1} X^T Y, \quad (17)$$

dónde:

- X es la matriz de datos de tamaño $n \times (m + 1)$.
- Y es el vector de valores observados de y .
- A es el vector de coeficientes $(a_0, a_1, \dots, a_m)$.

Suposiciones de la Regresión Lineal Múltiple. Para que la regresión lineal múltiple sea válida, deben cumplirse las siguientes condiciones $[?]$.

- Linealidad: La relación entre las variables es aproximadamente lineal.
- Independencia: No debe haber correlación entre los residuos.
- Homoscedasticidad: La varianza de los errores debe ser constante.
- No colinealidad: Las variables predictoras no deben estar altamente correlacionadas entre sí.
- Distribución normal de los errores: Los residuos deben seguir una distribución normal.

Error estándar de estimación. El **error estándar de estimación** $s_{y/x}$ mide la dispersión de los valores observados y_i en torno a la hiperplano de regresión. Se define como:

$$s_{y/x} = \sqrt{\frac{S_r}{n - (m + 1)}} \quad (18)$$

- Si $s_{y/x}$ es pequeño, los valores predichos $\hat{y}_i$ están cerca de los valores observados y_i , lo que indica un buen ajuste del modelo.
- Si $s_{y/x}$ es grande, la dispersión es alta, lo que sugiere que el modelo no explica bien la variabilidad de los datos.

2.2.4. Regresión No Lineal.

La **regresión no lineal** es un método estadístico que permite ajustar un modelo matemático cuando la relación entre las variables dependiente y e independientes x no es lineal en los parámetros [?, ?]. Esto significa que no se puede expresar como una combinación lineal de los coeficientes a estimar. Su forma general es:

$$y = f(x, a_0, a_1, \dots, a_m) + \varepsilon, \quad (19)$$

dónde $f(x, a_0, a_1, \dots, a_m)$ es una función no lineal en los coeficientes a_i , y ε es el término de error.

A diferencia de la regresión lineal, en la regresión no lineal **no se pueden obtener soluciones exactas mediante ecuaciones normales**, por lo que se emplean métodos iterativos para minimizar la suma de los errores cuadráticos [?]. $S = \sum_{i=1}^n (y_i - f(x_i, a_0, a_1, \dots, a_m))^2$.

Aproximación mediante Serie de Taylor. Para resolver este problema, una técnica común es **linealizar el modelo** usando una **expansión en serie de Taylor** alrededor de los valores actuales de los coeficientes ([?]) $a_m^{(j)}$:

$$f(x, a_m^{(j+1)}) \approx f(x, a_m^{(j)}) + \sum_{k=0}^m \frac{\partial f}{\partial a_k} \Delta a_k. \quad (20)$$

Aquí, Δa_k representa el cambio en los coeficientes en la iteración $j + 1$. Esta aproximación convierte el problema no lineal en un **sistema de ecuaciones lineales**.

Resolución mediante Mínimos Cuadrados No Lineales. Al reformular el problema en términos matriciales, se obtiene ([?]).

$$\mathbf{Z_J} \Delta \mathbf{A} = \mathbf{D}, \quad (21)$$

dónde:

- $\mathbf{Z_J}$ es la **matriz Jacobiana**, con elementos: $Z_{Jik} = \left. \frac{\partial f}{\partial a_k} \right|_{(x_i, a_k^{(j)})}$.
- $\mathbf{D}$ es el **vector de residuos**, dado por: $d_i = y_i - f(x_i, a_0, a_1, \dots, a_m)$.
- $\Delta \mathbf{A}$ es el vector de corrección de los coeficientes, que se obtiene resolviendo:

$$\Delta \mathbf{A} = (\mathbf{Z_J}^T \mathbf{Z_J})^{-1} \mathbf{Z_J}^T \mathbf{D}. \quad (22)$$

Este sistema permite encontrar ΔA , que se usa para actualizar los parámetros:

$$a_m^{(j+1)} = a_m^{(j)} + \Delta a_m. \quad (23)$$

Método Iterativo hasta la Convergencia. El procedimiento para resolver la regresión no lineal mediante mínimos cuadrados sigue los siguientes pasos ([?]):

1. Se eligen valores iniciales para los coeficientes $a_0, a_1, \dots, a_m$.
2. Se evalúan los valores predichos $f(x_i, a_m^{(j)})$ y la matriz Jacobiana $\mathbf{Z_J}$.
3. Se calcula $\Delta \mathbf{A}$ resolviendo $\mathbf{Z_J}^T \mathbf{Z_J} \Delta \mathbf{A} = \mathbf{Z_J}^T \mathbf{D}$.
4. Se actualizan los coeficientes con $a_m^{(j+1)} = a_m^{(j)} + \Delta a_m$.
5. Se repite el proceso hasta que la variación de los coeficientes sea menor que un umbral de tolerancia ε .

2.2.5. Linealización de Relaciones No Lineales.

Cuando un modelo no es lineal en sus parámetros, es posible aplicar transformaciones matemáticas para convertirlo en una forma lineal. Este proceso se conoce como **linealización** y permite resolver el problema utilizando el método de mínimos cuadrados de la regresión lineal. A continuación, se presentan algunos modelos no lineales comunes y su transformación.

Transformación de Modelos No Lineales.

- **Modelo Exponencial.** Se utiliza para representar crecimiento o decaimiento exponencial. Su forma general es:

$$y = a_0 - a_1 e^{-a_2 x}. \quad (24)$$

Para linealizar este modelo, tomamos el logaritmo natural en ambos lados:

$$\ln(-(y - a_0)) = \ln a_1 - a_2 x.$$

Esto permite reescribir la ecuación en forma lineal $Y = A + BX$, donde:

$$Y = \ln(-(y - a_0)), \quad A = \ln a_1, \quad B = -a_2, \quad X = x. \quad (25)$$

Luego, se aplica regresión lineal sobre Y y X para estimar A y B , y finalmente recuperar a_1 y a_2 .

- **Modelo de Potencia.** Se usa cuando la relación entre y y x sigue una ley de potencia:

$$y = a_0 x^{a_1}. \quad (26)$$

Aplicando logaritmos:

$$\ln y = \ln a_0 + a_1 \ln x.$$

Esto permite usar regresión lineal para encontrar $\ln a_0$ y a_1 , y luego recuperar a_0 mediante la exponencial.

- **Modelo Hiperbólico.** Se emplea para representar relaciones inversas:

$$y = \frac{a_0}{a_1 + x} \quad (27)$$

Tomamos el recíproco:

$$\frac{1}{y} = \frac{x}{a_0} + \frac{a_1}{a_0}.$$

Ahora, la relación es lineal, lo que permite estimar los parámetros mediante regresión lineal.

Resolución Mediante Mínimos Cuadrados. Una vez que los modelos han sido transformados en expresiones lineales, se pueden resolver usando **métodos de regresión lineal** (véase Sección ??).

Síntesis. La linealización de modelos no lineales permite utilizar técnicas de regresión lineal para estimar los parámetros de modelos complejos. Sin embargo, las transformaciones pueden afectar la distribución del error y deben aplicarse con cuidado. En casos donde la linealización no es viable, se recomienda utilizar métodos de ajuste no lineales como el **método de Levenberg-Marquardt**.

3. Resultados.

3.1. Ejercicio 1: Interpolación de Lagrange.

Se implementó la interpolación de Lagrange sobre el conjunto de datos experimentales, obteniéndose un polinomio interpolador que pasa exactamente por los puntos originales. La interpolación se evaluó en un conjunto de 100 puntos dentro del intervalo de análisis, generando la siguiente gráfica (véase la Figura ??) :

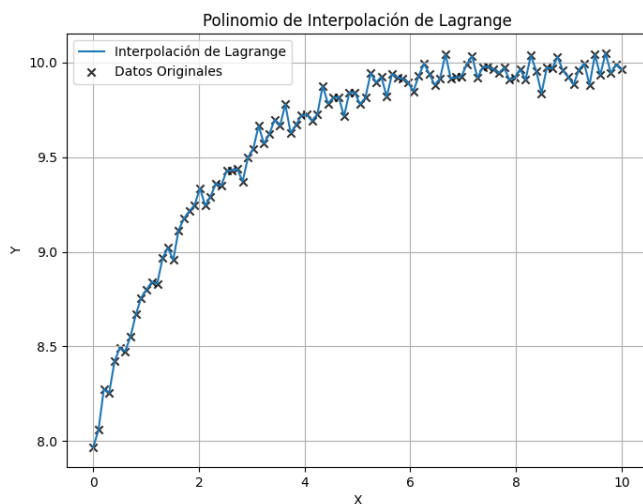


Figura 3: Interpolación de Lagrange aplicada a los datos experimentales.

Los resultados muestran que el polinomio ajusta con precisión los datos originales, reproduciendo exactamente los valores en los nodos dados. Para analizar con mayor detalle el comportamiento de la interpolación, se han identificado tres regiones dentro del intervalo (véase la Figura ??) :

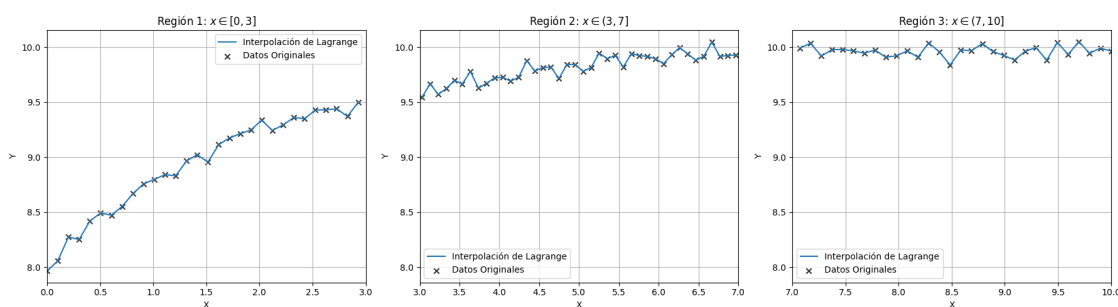


Figura 4: Análisis por Regiones de la Interpolación de Lagrange.

- **Región 1:** $x \in [0, 3]$. En esta zona, el polinomio de interpolación de Lagrange es una **aproximación adecuada**, ya que respeta la forma funcional de los datos sin introducir desviaciones artificiales. La función interpolada presenta continuidad y suavidad, reflejando fielmente la tendencia creciente de los datos por la distribución homogénea de los puntos.

- **Región 2:** $x \in (3, 7]$. Comienzan a observarse ligeras oscilaciones en la interpolación. Aunque la curva aún sigue la tendencia general de los datos, se evidencian **pequeñas fluctuaciones** en torno a los valores originales. Estas variaciones son resultado del comportamiento de los polinomios de alto grado, que intentan ajustarse de manera exacta a cada punto de la muestra. La **mayor densidad de puntos incrementa la acumulación de errores**, lo que genera desviaciones localizadas. Sin embargo, en términos generales, el polinomio interpolador sigue representando adecuadamente la función subyacente.
- **Región 3:** $x \in (7, 10]$. Las oscilaciones se hacen más evidentes, con la curva interpolada alternando entre valores superiores e inferiores a los puntos de la muestra. Este comportamiento es una **manifestación del Fenómeno de Runge**, donde el polinomio interpolador de alto grado introduce inestabilidades debido al **sobreajuste de los datos**. En esta zona, las fluctuaciones locales de los valores originales amplifican los efectos de interpolación, lo que da lugar a una pérdida de precisión en la representación de la tendencia general de la función.

Los resultados confirman que la interpolación de Lagrange es efectiva para ajustar los datos experimentales, proporcionando una representación continua y suave. No obstante, en algunas zonas del dominio, especialmente a medida que se incrementa el número de puntos, se pueden notar fluctuaciones leves que podrían impactar la estabilidad de la interpolación. Este aspecto se analizará en detalle en la sección de **Discusión** (véase Sección ??).

3.2. Ejercicio 2: Interpolación mediante Splines Cúbicos.

Se aplicó la interpolación con Splines Cúbicos al conjunto de datos experimentales, obteniendo una curva suavizada que conecta los puntos originales de manera continua y con una transición suave en cada subintervalo (véase la Figura ??).

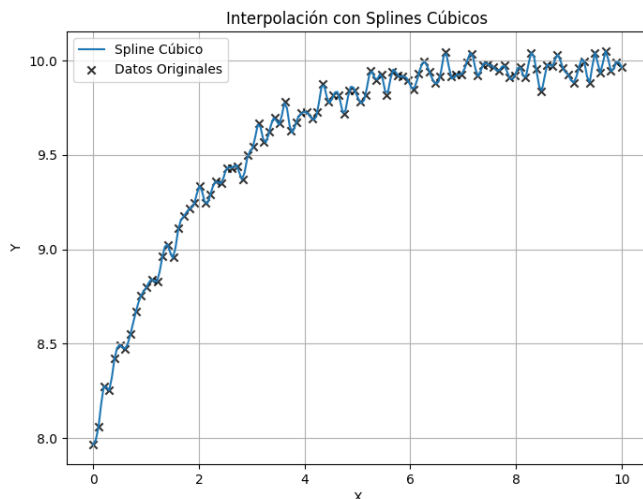


Figura 5: Interpolación con Splines Cúbicos aplicada a los datos experimentales.

Los resultados muestran que los Splines Cúbicos **evitan las oscilaciones de alto grado** en los extremos y permiten un **ajuste más estable** a lo largo del dominio.

Para analizar en detalle el comportamiento del Spline Cúbico, se observa lo siguiente en **diferentes regiones** del intervalo (véase la Figura ??):

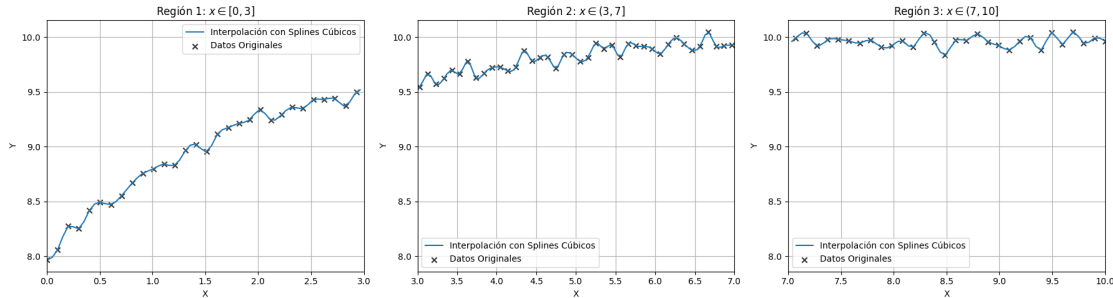


Figura 6: Análisis por Regiones de la Interpolación mediante Splines Cúbicos.

- **Región 1:** $x \in [0, 3]$. La interpolación con Splines Cúbicos presenta un **ajuste preciso y suave** a los datos originales. La continuidad en la primera y segunda derivada garantiza una transición uniforme entre segmentos, evitando oscilaciones indeseadas. La baja dispersión en los puntos de datos y su disposición equiespaciada favorecen la estabilidad de la interpolación, asegurando una representación fiel de la tendencia general.
- **Región 2:** $x \in (3, 7]$. Se mantiene una **aproximación estable**, pero se observan **pequeñas variaciones** en torno a los valores originales. Estas fluctuaciones no corresponden a oscilaciones descontroladas, sino a la flexibilidad inherente del spline para adaptarse a los cambios locales sin comprometer la suavidad global.
- **Región 3:** $x \in (7, 10]$. La interpolación sigue la distribución de los datos **sin introducir el Fenómeno de Runge**. El Spline Cúbico minimiza los errores de interpolación al fragmentar el dominio en intervalos locales. La estabilidad numérica se mantiene, y la precisión en los valores interpolados es mayor debido a la restricción de continuidad en las derivadas. Esto demuestra que los Splines Cúbicos son una alternativa eficiente y robusta para la interpolación en conjuntos de datos densos o con variabilidad localizada.

Los resultados confirman que la interpolación con Splines Cúbicos proporciona un ajuste más estable en comparación con la interpolación de Lagrange, especialmente en los extremos del intervalo. En la sección de **Discusión** (véase Sección ??), se compararán ambas técnicas en términos de estabilidad, precisión y aplicabilidad en distintos contextos.

3.3. Ejercicio 3: Ajuste por Regresión Lineal.

Se han obtenido dos tipos de ajuste para los datos experimentales: mediante una **regresión polinómica de distintos grados** (1, 2, 3 y 4) utilizando el método de mínimos cuadrados, así como un ajuste utilizando una **función potencial** en una escala log-log: $y = a_0x^{a_1}$. A partir de las gráficas obtenidas, se pueden extraer las siguientes observaciones clave:

Regresión polinómica.

La Figura ?? muestra los resultados obtenidos para los distintos grados polinomiales en comparación con los datos originales.

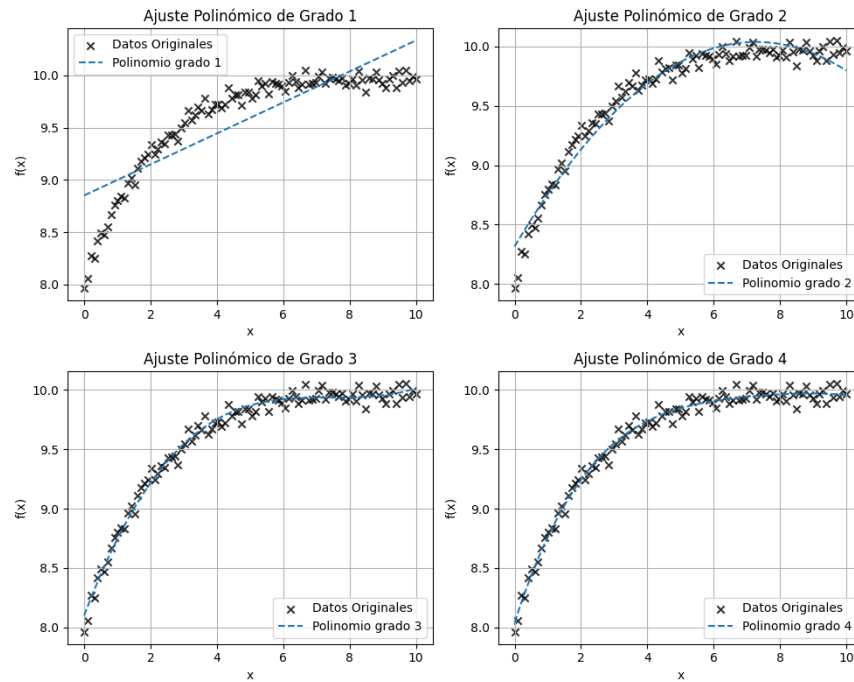


Figura 7: Ajuste de los datos mediante regresión polinómica de distintos grados.

Las ecuaciones de las rectas obtenidas y el coeficiente de determinación R^2 para cada ajuste polinómico se presentan en la Tabla ??.

Grado	Ecuación del ajuste	R^2
1	$y = 8,852 + 0,148x$	0,722340
2	$y = 8,316 + 0,472x - 0,032x^2$	0,958882
3	$y = 8,100 + 0,738x - 0,099x^2 + 0,004x^3$	0,987903
4	$y = 8,042 + 0,860x - 0,154x^2 + 0,013x^3 - 0,0004x^4$	0,989690

Tabla 1: Ecuaciones obtenidas y coeficiente de determinación R^2 para regresión polinómica.

El ajuste por regresión polinómica muestra una mejora progresiva en la precisión conforme aumenta el grado del polinomio.

- **Polinomio de grado 1.** La ecuación obtenida ($y = 8,852 + 0,148x$) presenta un coeficiente de determinación $R^2 = 0,722$ lo que indica un **ajuste deficiente**. Este modelo lineal no logra capturar la curvatura de los datos, lo que resulta en una **subestimación sistemática** en los valores intermedios y una **sobreestimación** en los extremos.
- **Polinomio de grado 2.** El modelo cuadrático $y = 8,316 + 0,472x - 0,032x^2$ mejora significativamente la precisión, alcanzando un $R^2 = 0,959$. Este polinomio comienza a **capturar la curvatura** de la tendencia, proporcionando un **ajuste más preciso en la zona central**, aunque aún presenta **ciertas desviaciones** en los extremos del intervalo.
- **Polinomio de grado 3.** El ajuste mejora con el polinomio cúbico $y = 8,100 + 0,738x - 0,099x^2 + 0,004x^3$ que logra un $R^2 = 0,988$. Se observa que este modelo **ajusta con precisión** tanto la parte creciente como la estabilización de los datos en valores altos, reduciendo las desviaciones observadas en grados menores.
- **Polinomio de grado 4.** Este modelo $y = 8,042 + 0,860x - 0,154x^2 + 0,013x^3 - 0,0004x^4$ presenta el mejor ajuste con $R^2 = 0,99$. Sin embargo, el **incremento en precisión respecto al modelo de grado 3 es marginal**, lo que sugiere que aumentar aún más el grado del polinomio podría no ser beneficioso y, en cambio, podría llevar a **sobreajuste**.

Estos resultados evidencian que los **modelos de grado bajo no son suficientes para capturar la estructura** de los datos, mientras que los **modelos de grado intermedio**, como el de grado 3, proporcionan un **equilibrio** adecuado entre precisión y estabilidad. Para analizar en mayor profundidad la implicación de estos resultados, se remite al apartado de **Discusión** (véase Sección ??).

Función potencial.

La escala log-log $y = a_0x^{a_1}$, revela relaciones de crecimiento lento y tendencias asíntoticas. La Figura ?? compara el ajuste de la función potencial con los datos originales.

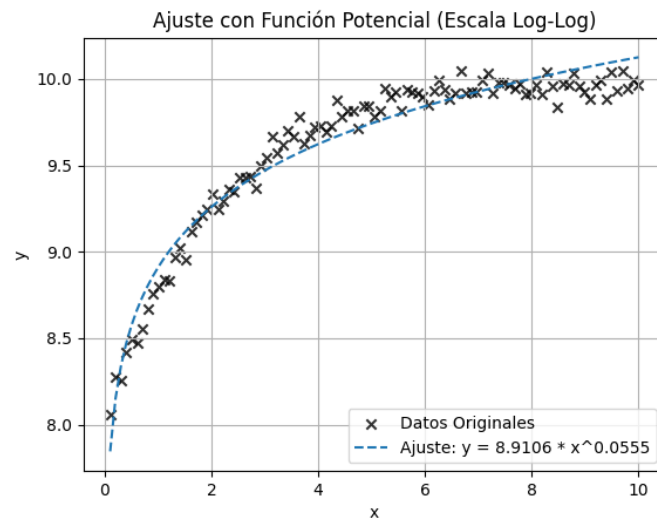


Figura 8: Ajuste de los datos mediante función potencial.

El ajuste mediante una función potencial se realizó en una escala log-log, obteniendo la ecuación:

$$y = 8,911 \cdot x^{0,056} \quad (28)$$

En la **parte baja del dominio**, el modelo **captura** de manera efectiva la **curvatura inicial** de los datos, mostrando una buena concordancia con los valores originales. A medida que x **aumenta**, la función de ajuste mantiene una **representación aceptable** de la tendencia general, aunque pueden observarse **pequeñas desviaciones** en los valores más altos. Sin embargo, la **forma global del ajuste es consistente con la estructura de los datos**, proporcionando una interpolación suavizada sin oscilaciones abruptas.

En términos de **precisión**, el modelo potencial presenta un coeficiente de determinación de $R^2 = 0,9547$, lo que indica un buen nivel de ajuste, aunque inferior a los obtenidos con **polinomios de grado 3 y 4**.

Para evaluar la idoneidad de este modelo, es relevante compararlo con la regresión polinómica vista previamente. Esta comparación se abordará en la **Discusión** (véase Sección ??).

3.4. Ejercicio 4: Ajuste por Regresión No Lineal.

Se realizó un ajuste a los datos experimentales utilizando distintos modelos de regresión no lineal, evaluando su desempeño en la aproximación de la relación entre las variables. Los modelos considerados fueron:

- **Modelo Exponencial:** $f(x) = a_0 - a_1 e^{-a_2 x}$.
- **Modelo Lineal-Exponencial:** $f(x) = a_0 x - a_1 e^{-a_2 x}$.
- **Modelo Cuadrático-Exponencial:** $f(x) = a_0 x^2 - a_1 e^{-a_2 x}$.

La Figura ?? muestra la comparación entre los datos originales y los ajustes obtenidos con cada modelo.

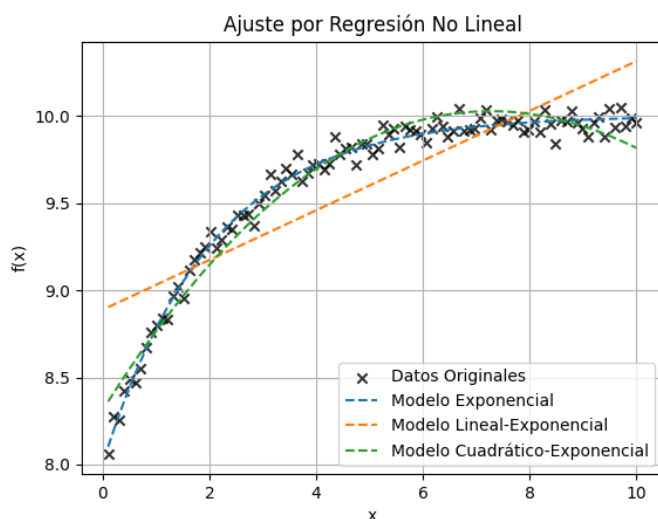


Figura 9: Ajuste de los datos mediante modelos de regresión no lineal.

Los coeficientes de ajuste obtenidos para cada modelo, junto con sus valores de R^2 , se presentan en la Tabla ??.

Modelo	a_0	a_1	a_2	R^2	Ecuación
Exponencial	10,0019	1,9965	0,4918	0,991	$y = 10,0019 - 1,9965e^{-0,4918x}$
Lineal-Exponencial	0.1426	-8,8885	-0,000003	0,724	$y = 0,1426x - 8,8885e^{-0,000003x}$
Cuadrático-Exponencial	-0,0518	-8,3125	-0,0590	0,965	$y = -0,0518x^2 - 8,3125e^{-0,0590x}$

Tabla 2: Coeficientes obtenidos y coeficiente de determinación R^2 para regresión no lineal.

De los resultados obtenidos se sacan las siguientes conclusiones:

- **Modelo exponencial:** Representa un crecimiento que se desacelera a medida que x aumenta, capturando adecuadamente la tendencia general de los datos en la parte inicial del intervalo. Sin embargo, en la región superior, el modelo muestra un error creciente para valores altos de x , lo que sugiere que su capacidad de predicción es más limitada en estos casos. A pesar de ello, su alto R^2 indica que sigue siendo el modelo más preciso en términos generales.
- **Modelo lineal-exponencial:** Presenta un ajuste menos preciso en comparación con los demás modelos, con un R^2 considerablemente menor. En la zona intermedia del intervalo, la pendiente lineal no logra capturar correctamente la curvatura de los datos, lo que lleva a **desviaciones significativas** en la predicción. Además, la **contribución del término exponencial** es mínima, haciendo que el modelo se comporte casi como una **regresión lineal simple** y perdiendo su capacidad de representar correctamente la tendencia de los datos.
- **Modelo cuadrático-exponencial:** Es el que mejor se ajusta a los datos a lo largo de todo el intervalo, manteniendo una buena correspondencia con la tendencia original. Su R^2 es alto, aunque ligeramente inferior al del modelo exponencial puro. La inclusión del término cuadrático permite capturar mejor la curvatura de los datos en la región inicial, reduciendo el error en comparación con los otros modelos. Sin embargo, en valores altos de x , su precisión no mejora significativamente con respecto al modelo exponencial.

Los resultados indican que, aunque el **modelo exponencial** es el **más preciso** en términos de R^2 , el **modelo cuadrático-exponencial** ofrece un **compromiso razonable**, logrando una representación **equilibrada** en todas las regiones del intervalo. Para comprender mejor las diferencias entre estos modelos y su aplicabilidad, se realiza un análisis comparativo en el apartado de **Discusión**(véase Sección ??).

4. Discusión.

4.1. Ejercicio 1: Interpolación de Lagrange.

La interpolación de Lagrange es **precisa en intervalos pequeños** con puntos distribuidos de manera uniforme, pero su **estabilidad disminuye conforme aumenta el número de nodos** o la densidad de datos. En particular, la última región mostrada en el apartado de **Resultados** (véase Sección ??), mostró que los polinomios de alto grado tienden a oscilar en los extremos del intervalo, reduciendo la confiabilidad de la aproximación.

Eficiencia computacional.

Desde el punto de vista **computacional**, la interpolación de Lagrange presenta una complejidad de $O(n^2)$ en la construcción del polinomio y $O(n)$ en su evaluación, ya que requiere evaluar productos de múltiples términos. Esto provoca que para **conjuntos de datos grandes, sea computacionalmente costoso**. Métodos alternativos, tales como los **Splines Cúbicos** (véase Sección ??), ofrecen mayor eficiencia al dividir el dominio en segmentos, reduciendo el número de operaciones necesarias.

Precisión y capacidad de ajuste.

En términos de **precisión**, Lagrange es exacto en los nodos de interpolación, pero propenso a **errores en regiones con alta variabilidad** o con datos no equiespaciados. Por el contrario, los **Splines Cúbicos suavizan la interpolación** sin forzar un ajuste exacto, reduciendo oscilaciones y mejorando la representación global de los datos.

Estabilidad numérica y robustez.

En cuanto a **estabilidad**, los **polinomios de alto grado son sensibles a pequeñas perturbaciones** en los datos, lo que puede generar **oscilaciones no deseadas**. Métodos como la **interpolación de Newton** (véase Sección ??) y los **Splines Cúbicos** distribuyen mejor los errores y evitan el crecimiento descontrolado del polinomio, lo que los hace más adecuados para aplicaciones prácticas que requieren robustez en la aproximación.

4.2. Ejercicio 2: Interpolación mediante Splines Cúbicos.

Los Splines Cúbicos destacan por su capacidad para **generar interpolaciones suaves y estables**, lo que los convierte en una opción óptima en problemas donde la **continuidad en las derivadas es crucial**. Su formulación basada en polinomios por tramos evita los inconvenientes de los polinomios de alto grado, distribuyendo la interpolación de manera más controlada y evitando oscilaciones innecesarias.

El **análisis comparativo** con la interpolación de **Lagrange** demuestra que los **Splines Cúbicos** son una opción más estable y eficiente en **escenarios con alta densidad** de puntos. Mientras que el polinomio de Lagrange puede generar oscilaciones en los extremos del intervalo, el Spline Cúbico evita estos problemas al dividir el dominio en segmentos locales.

Eficiencia computacional.

Desde una perspectiva **computacional**, la eficiencia de los **Splines Cúbicos** es notable. La necesidad de resolver un sistema tridiagonal de ecuaciones lineales implica una complejidad de $O(n)$, lo que resulta más eficiente en comparación con el $O(n^2)$ de la **interpolación de Lagrange** (véase Sección ??). Esto los

hace adecuados para conjuntos de datos grandes, dónde la velocidad de cálculo y la estabilidad numérica son factores determinantes.

Precisión y capacidad de ajuste.

En términos de **precisión**, los **Splines Cúbicos** logran un **ajuste exacto** en los nodos de interpolación sin incurrir en sobreajustes globales. A diferencia de métodos que utilizan un único polinomio de alto grado, los Splines aseguran que la interpolación siga fielmente la tendencia de los datos sin introducir oscilaciones artificiales. En particular, su capacidad para conservar la continuidad en la primera y segunda derivada permite que la interpolación sea más natural y representativa en aplicaciones dónde la suavidad es un requisito clave.

Estabilidad numérica y robustez.

En cuanto a **estabilidad**, los **Splines Cúbicos** son una alternativa robusta frente a otras metodologías de interpolación, ya que **evitan la amplificación de errores** en los extremos y permiten un **control** más refinado de la curvatura de la función interpolada. Esto es especialmente relevante en problemas dónde los datos presentan variaciones localizadas, ya que la interpolación segmentada minimiza los errores acumulativos y mantiene la coherencia en la aproximación.

Comparación con la interpolación de Lagrange.

Comparando con la **interpolación de Lagrange**, los resultados obtenidos confirman que los **Splines Cúbicos** son preferibles en situaciones dónde la **estabilidad** y la **eficiencia computacional** son **prioritarias**. Mientras que **Lagrange** puede generar oscilaciones indeseadas en regiones con alta densidad de puntos, los splines cúbicos logran un ajuste más controlado y menos susceptible a fluctuaciones. En consecuencia, su uso es recomendable en aplicaciones dónde la interpolación debe **preservar la suavidad y garantizar precisión en intervalos amplios** sin comprometer la estabilidad numérica (véase la Figura ??).

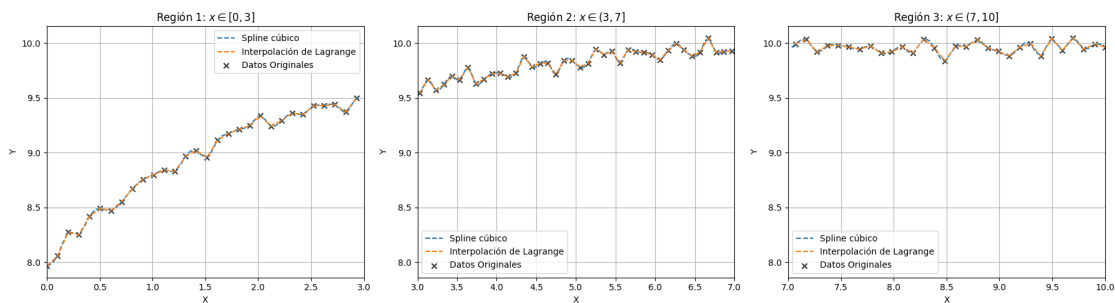


Figura 10: Comparación por regiones: Interpolación de Lagrange y Splines Cúbicos.

4.3. Ejercicio 3: Ajuste por Regresión Lineal.

Regresión polinómica.

- **Relación entre el grado del polinomio y la precisión del ajuste:** El **coeficiente de determinación** muestra un **aumento** progresivo a medida que el **grado del polinomio crece**, lo que indica que modelos de mayor orden pueden representar mejor la variabilidad de los datos. Sin embargo, la mejora entre los grados 3 y 4 es mínima, lo que sugiere que el **modelo de grado 3 es suficiente** para capturar la tendencia sin incurrir en complejidad innecesaria.
- **Sobreajuste y estabilidad del modelo:** Aumentar excesivamente el grado del polinomio puede llevar a un problema de sobreajuste, donde el modelo se ajusta demasiado a los datos de entrenamiento y pierde capacidad de generalización. Aunque el polinomio de grado 4 presenta un ajuste superior, su complejidad adicional podría no justificar el pequeño incremento en R^2 . En aplicaciones prácticas, se recomienda optar por **modelos de grado intermedio para evitar oscilaciones**.
- **Interpretabilidad y simplicidad del modelo:** Los **polinomios de grado bajo**, proporcionan una **interpretación más sencilla** de la relación entre las variables, pero **no capturan la curvatura de los datos**. A medida que aumenta el grado, el modelo se vuelve más flexible, pero también más difícil de interpretar. Se debe encontrar un equilibrio entre precisión y simplicidad.
- **Eficiencia computacional:** Desde el punto de vista computacional, la regresión polinómica requiere resolver un sistema de ecuaciones normales, cuya **complejidad depende del grado del polinomio** y del método de solución utilizado. Para grados bajos, el cálculo es rápido y eficiente, pero a medida que el grado aumenta, los cálculos pueden volverse más costosos y propensos a inestabilidad numérica.

Función potencial.

- **Crecimiento desacelerado:** La ecuación obtenida indica que la variable dependiente crece con x , pero a un ritmo decreciente, lo que sugiere una **convergencia progresiva en valores altos de x** .
- **Escala log-log:** La transformación logarítmica utilizada en este ajuste permite **identificar patrones de crecimiento lento o comportamiento asintótico** en los datos. Esto es útil cuando se sospecha que la relación entre variables sigue una ley de potencias en lugar de una función polinómica.
- **Capacidad predictiva:** Si el objetivo es modelar tendencias generales y extrapolar datos fuera del intervalo analizado, el ajuste con función potencia es una opción adecuada, ya que mantiene estabilidad en la representación de los datos sin introducir fluctuaciones artificiales.
- **Eficiencia computacional:** El ajuste potencia requiere únicamente una regresión lineal en la escala log-log, lo que reduce su complejidad computacional a $O(n)$. Esto lo hace **Más eficiente que los polinomios de alto grado**, cuya solución requiere cálculos más costosos.

Comparación de Ajustes.

- **Precisión vs. estabilidad:** La regresión polinómica ofrece un ajuste más preciso dentro del intervalo de datos, alcanzando valores de R^2 superiores al ajuste potencial. Sin embargo, los **polinomios de grado alto pueden generar sobreajuste**, capturando ruido en los datos en lugar de representar la tendencia general. En cambio, la **función potencial** proporciona un ajuste más **estable y sin oscilaciones** indeseadas.

- **Extrapolación y generalización:** Si el propósito es simplemente **interpol**ar dentro del conjunto de datos, un **polinomio** de grado intermedio o alto puede ser adecuado. No obstante, si se requiere **extrapolar**, la **función potencial es una opción más confiable**, ya que sigue una tendencia más predecible sin las oscilaciones que pueden aparecer en polinomios de alto grado.
- **Eficiencia computacional:** Mientras que los **polinomios** requieren la resolución de sistemas de ecuaciones de mayor complejidad $O(n^3)$ en su versión estándar, la **función potencial** reduce el problema a una regresión lineal en escala log-log, lo que implica menor costo computacional y una mayor estabilidad numérica.

4.4. Ejercicio 4: Ajuste por Regresión No Lineal.

Comparación entre los modelos.

- **Precisión vs. estabilidad:** El **modelo exponencial** es el que presenta el **mayor R^2** , lo que indica que proporciona la **mejor precisión** dentro del intervalo de datos. Sin embargo, su **desviación en valores altos** de x puede ser un **inconveniente** si se requiere una **representación uniforme** en todo el dominio. El modelo **cuadrático-exponencial**, aunque con un R^2 ligeramente menor, logra una **mejor estabilidad en el ajuste**, lo que lo hace más adecuado en escenarios donde se busca una **interpolación más homogénea**.
- **Eficiencia del modelo lineal-exponencial:** La baja precisión de este modelo sugiere que la **combinación de términos lineales y exponenciales no es efectiva** en este caso. La contribución del término exponencial es prácticamente despreciable, lo que hace que el modelo actúe como una regresión lineal simple sin capturar la curvatura real de los datos.
- **Capacidad de extrapolación:** En aplicaciones donde se requiera extrapolar datos más allá del intervalo analizado, la selección del modelo adecuado es crucial. Si bien el modelo exponencial captura bien la tendencia inicial, su error creciente en valores altos puede limitar su uso en predicciones a largo plazo. El **modelo cuadrático-exponencial podría ofrecer un mejor ajuste** en estas condiciones, aunque su estructura más compleja puede hacer que la extrapolación sea menos intuitiva.

Aplicabilidad y selección del modelo adecuado.

- Si se busca **máxima precisión** en la interpolación dentro del intervalo de datos, el **modelo exponencial** es la mejor opción.
- Si se **prioriza la estabilidad** del ajuste en todo el dominio, el **modelo cuadrático-exponencial** puede ser más adecuado.
- El **modelo lineal-exponencial es el menos recomendable**, ya que no captura correctamente la estructura de los datos.

Desde una perspectiva práctica, la elección del modelo **depende del propósito del análisis**. Para describir relaciones que sigan un comportamiento de crecimiento desacelerado, el modelo exponencial es una opción sólida. Sin embargo, si se requiere una mejor estabilidad en toda la región de análisis, el modelo cuadrático-exponencial puede proporcionar un mejor equilibrio entre precisión y generalización.

5. Conclusiones.

La **interpolación de Lagrange** es precisa en regiones con puntos bien distribuidos, pero sufre inestabilidad en los extremos debido al **Fenómeno de Runge**, lo que la hace **poco recomendable para intervalos amplios**. Su alto costo computacional limita su uso en grandes volúmenes de datos.

Los **Splines Cúbicos** ofrecen una alternativa más estable y eficiente a la **interpolación de Lagrange**, garantizando suavidad y continuidad sin oscilaciones no deseadas. Son ideales para interpolaciones donde se requiere estabilidad en todo el intervalo y menor complejidad computacional.

La **regresión polinómica** de grado intermedio proporciona un **equilibrio** entre precisión y estabilidad, evitando el **sobreajuste** de polinomios de **grado alto**. Sin embargo, polinomios elevados pueden generar oscilaciones en la extrapolación y mayor costo computacional. El **ajuste con función potencial** es eficiente y adecuado para datos con crecimiento desacelerado. Aunque su precisión es menor que la polinómica, **evita oscilaciones extremas** y permite una extrapolación más estable con menor costo computacional.

En la **regresión no lineal**, el **modelo exponencial** logra el mejor ajuste, aunque con errores crecientes en valores altos de x . El **modelo cuadrático-exponencial** mejora la estabilidad sin superar la precisión del exponencial puro, mientras que el **modelo lineal-exponencial** no logra captar la curvatura de los datos.

En términos generales, para **interpolaciones precisas** dentro del intervalo, los **Splines Cúbicos** y los **polinomios de grado intermedio** son ideales. Para **estabilidad y extrapolación**, la **función potencia** y el **modelo exponencial** son mejores opciones. En términos de eficiencia, los **Splines Cúbicos** y la **función potencia** destacan sobre métodos de mayor complejidad.

6. Referencias.

- [1] Ingeniería UNAM. (s.f.). *Interpolación de Lagrange*. Recuperado de https://www.ingenieria.unam.mx/pinilla/PE105117/pdfs/tema4/4-1_lagrange.pdf
- [2] Díaz, J. (s.f.). *Interpolación de Lagrange*. Universitat de València. Recuperado de https://www.uv.es/diazj/cn_tema5.pdf
- [3] ANIEL. (s.f.). *Curso de Métodos Numéricos*. Recuperado de https://aniei.org.mx/paginas/uam/CursoMN/curso_mn_07.html
- [4] Díaz, J. (s.f.). *Splines Cúbicos*. Universitat de València. Recuperado de <https://www.uv.es/diaz/mn/node40.html>
- [5] Universidad de Sevilla. (s.f.). *Interpolación por Splines*. Recuperado de https://biblus.us.es/bibing/proyectos/abreproy/3989/fichero/MEMORIA%252F13_ANEX07.pdf
- [6] UNAM. (s.f.). *Métodos Numéricos en Ingeniería*. Recuperado de <https://www.paginaspersonales.unam.mx/app/webroot/files/4676/Asignaturas/1457/Archivo1.3288.pdf>
- [7] Universitat de Barcelona. (s.f.). *Interpolación por Splines*. Recuperado de <https://diposit.ub.edu/dspace/bitstream/2445/122512/2/memoria.pdf>
- [8] UNAM. (s.f.). *Regresión Polinómica*. Recuperado de https://www.ccg.unam.mx/~vinuesa/R4biosciences/docs/Tema9_regresion.html
- [9] Universidad de Murcia. (s.f.). *Regresión Polinómica y Cuadrados Mínimos*. Recuperado de <https://webs.um.es/mhcifre/apuntes/Capitulo4.pdf>
- [10] Díaz, J. (s.f.). *Regresión Polinómica*. Universitat de València. Recuperado de https://www.uv.es/~diazj/cn_tema7.pdf
- [11] Universidad del País Vasco. (s.f.). *Regresión Polinómica*. Recuperado de <http://www.sc.ehu.es/sbweb/fisica3/datos/regresion/regresion.html>
- [12] Universidad de Buenos Aires. (s.f.). *Regresión Lineal por Cuadrados Mínimos*. Recuperado de <https://materias.df.uba.ar/mytda2020c2/files/2013/03/RegresionLinealPorCuadradosMinimos.pdf>
- [13] Universidad de Zaragoza. (s.f.). *Modelos de Regresión No Lineal*. Recuperado de https://ocw.unizar.es/ocw/pluginfile.php/2991/course/section/908/Tema%208_Modelos%20de%20Regresi%C3%B3n%20No%20lineal.pdf
- [14] Universidad de Alicante. (s.f.). *Regresiones Lineales y No Lineales*. Recuperado de <https://rua.ua.es/dspace/bitstream/10045/16373/4/Microsoft%20Word%20-%204.REGRESIONES%20LINEALES%20Y%20NO%20LINEALES.pdf>

7. Anexos.

El código fuente de estas implementaciones está disponible en el siguiente repositorio de GitHub: [GitHub - mgonzalz/sn-talleres](#).

7.1. Inicialización.

En esta sección, se define una función para leer los datos desde un archivo de texto (‘.txt’) utilizando NumPy. Luego, los datos se representan en un gráfico de dispersión mediante Matplotlib.

Listing 1: Lectura y graficación de datos.

```

1 # Funcion para leer los datos desde un archivo txt.
2 def leer_datos(archivo):
3     datos = np.loadtxt(archivo)
4     x = datos[:, 0]
5     y = datos[:, 1]
6     return x, y
7
8 # Graficacion de los datos originales.
9 plt.figure(figsize=(8, 6))
10 plt.scatter(x_data, y_data, color='black', marker='x', alpha=0.8)
11 plt.xlabel("X")
12 plt.ylabel("Y")
13 plt.title("Datos Originales")
14 plt.grid(True)
15 plt.savefig("../figures/informe-01/datos.png")
16 plt.show()

```

7.2. Función de Interpolación de Lagrange.

En esta sección, se implementa la interpolación de Lagrange, calculando la base $L_i(x)$ y evaluando el polinomio interpolador.

Listing 2: Implementación de la Interpolación de Lagrange.

```

1 # Funcion para calcular los coeficientes de Lagrange L_i(x).
2 def lagrange_basis(x, x_data, i):
3     n = len(x_data)
4     L_i = 1
5     for j in range(n):
6         if j != i:
7             L_i *= (x - x_data[j]) / (x_data[i] - x_data[j])
8     return L_i
9
10 # Funcion para calcular el polinomio de interpolación de Lagrange.
11 def polinomio_lagrange(x, x_data, y_data):
12     n = len(x_data)
13     resultado = 0
14     for i in range(n):
15         resultado += y_data[i] * lagrange_basis(x, x_data, i)
16     return resultado

```

7.3. Función de Interpolación mediante Splines Cúbicos.

En esta sección, se implementa la interpolación por splines cúbicos naturales, construyendo los polinomios por tramos y resolviendo un sistema tridiagonal para los coeficientes.

Listing 3: Implementación de la Interpolación por Splines Cúbicos.

```

1 # Construcción del spline c bico natural. Devuelve los coeficientes de los polinomios por
  tramos.
2 def spline_cubico(x_data, y_data):
3     n = len(x_data) - 1 # Numero de intervalos.
4     h = np.diff(x_data) # Tama o de cada subintervalo.
5     b = np.diff(y_data) / h # Diferencias divididas.
6
7     # Matriz tridiagonal y vector de terminos independientes.
8     A = np.zeros((n + 1, n + 1))
9     b_vec = np.zeros(n + 1)
10
11     # Condiciones de frontera naturales.
12     A[0, 0] = 1
13     A[n, n] = 1
14
15     for i in range(1, n):
16         A[i, i - 1] = h[i - 1]
17         A[i, i] = 2 * (h[i - 1] + h[i])
18         A[i, i + 1] = h[i]
19         b_vec[i] = 3 * (b[i] - b[i - 1])
20
21     # Resolucion del sistema tridiagonal para obtener segunda derivada (c coeficientes)
22     c = np.linalg.solve(A, b_vec)
23
24     # Calcular coeficientes a, b, c, d para cada subintervalo
25     a = y_data[:-1]
26     d = np.diff(c) / (3 * h)
27     b = b - h * (2 * c[:-1] + c[1:]) / 3
28
29     return a, b, c[:-1], d, x_data
30
31
32 # Evaluacion del spline cubico en un conjunto de puntos x.
33 def evaluar_spline(x, x_data, a, b, c, d):
34     n = len(x_data) - 1
35     y_vals = np.zeros_like(x)
36
37     for i in range(n):
38         indices = (x >= x_data[i]) & (x <= x_data[i + 1])
39         dx = x[indices] - x_data[i]
40         y_vals[indices] = a[i] + b[i] * dx + c[i] * dx**2 + d[i] * dx**3
41
42     return y_vals

```

7.4. Ajuste Polinómico por Mínimos Cuadrados.

En esta sección, se implementa el ajuste de un polinomio mediante el método de mínimos cuadrados, utilizando la matriz de Vandermonde para la estimación de coeficientes y la resolución de ecuaciones normales.

Listing 4: Implementación del Ajuste Polinómico por Mínimos Cuadrados.

```

1 # Funcion para ajustar un polinomio de mediante m nimos cuadrados.
2 def polynomial_regression(x, y, degree):
3     X = np.vander(x, degree + 1, increasing=True) # Matriz de Vandermonde.
4     coeffs = np.linalg.inv(X.T @ X) @ X.T @ y # Resolver ecuaci n normal.
5     return coeffs
6
7 # Funcion para evaluar un polinomio en los puntos dado un conjunto de coeficientes.
8 def evaluate_polynomial(coeffs, x_vals):
9     y_vals = np.zeros_like(x_vals)
10    for i, coef in enumerate(coeffs):
11        y_vals += coef * (x_vals ** i) # Evaluaci n manual del polinomio.
12    return y_vals

```

7.5. Ajuste Logarítmico por Mínimos Cuadrados

En esta sección, se implementa el ajuste de una función potencial mediante el método de mínimos cuadrados, utilizando una transformación logarítmica para convertir el modelo en una regresión lineal en la escala log-log.

Listing 5: Implementación del Ajuste Logarítmico por Mínimos Cuadrados.

```

1 '''
2 # Funcion potencial.
3 def power_law(x, a0, a1):
4     return a0 * x**a1
5
6 # Filtrar datos para evitar problemas con log(0)
7 valid_indices = x_data > 0
8 x_nonzero = x_data[valid_indices]
9 y_nonzero = y_data[valid_indices]
10
11 # Transformacion logaritmica.
12 log_x_nonzero = np.log(x_nonzero)
13 log_y_nonzero = np.log(y_nonzero)
14
15 # Ajustar una regresion lineal en la escala log-log
16 coeffs = polynomial_regression(log_x_nonzero, log_y_nonzero, degree=1)
17
18 # Extracci n de coeficientes
19 a1_manual = coeffs[1] # Pendiente
20 log_a0_manual = coeffs[0] # Intercepto en la escala log-log
21 a0_manual = np.exp(log_a0_manual) # Volver a la escala original
22 '''

```

7.5.1. Ajuste No Lineal: Métodos Automático y Manual.

Para ajustar modelos no lineales a los datos disponibles, se proponen dos enfoques:

- **Método Automático con `curve_fit`:** Se utiliza la función `curve_fit` de SciPy, que aplica internamente optimización no lineal basada en mínimos cuadrados. Es una opción rápida y eficiente para ajustar modelos sin necesidad de derivadas analíticas.
- **Método Manual con Gauss-Newton:** Se implementa el ajuste mediante el método de Gauss-Newton, calculando explícitamente la matriz Jacobiana para mejorar la convergencia. Este enfoque es útil cuando se requiere un control más preciso sobre el ajuste.

Dado que los datos de y contienen ruido, es importante modificar el ajuste para garantizar una mejor convergencia y minimizar la influencia de valores atípicos. A continuación, se presentan ambas implementaciones:

Listing 6: Ajuste No Lineal Automático con `curve_fit`.

```

1 # Modelos a resolver.
2 modelos = {
3     "Modelo_Exponencial": modelo_exponencial,
4     "Modelo_Lineal-Exponencial": modelo_lineal_exponencial,
5     "Modelo_Cuadrático-Exponencial": modelo_cuadratico_exponencial
6 }
7
8 # Parametros iniciales para cada modelo.
9 parametros_iniciales = {
10     "Modelo_Exponencial": [10, 5, 0.5],
11     "Modelo_Lineal-Exponencial": [1, 5, 0.5],
12     "Modelo_Cuadrático-Exponencial": [1, 5, 0.5]
13 }
14
15 # Diccionario para almacenar los parámetros ajustados.
16 parametros_ajustados = {}
17
18 # Ajuste de los modelos.
19 for nombre, modelo in modelos.items():
20     params_opt, _ = curve_fit(modelo, x_nonzero, y_nonzero, p0=parametros_iniciales[nombre])
21     # Ajuste con valores iniciales
22     parametros_ajustados[nombre] = params_opt

```

Listing 7: Ajuste No Lineal Manual con Gauss-Newton.

```

1 def modelo_exponencial(x, a0, a1, a2):
2     return a0 - a1 * np.exp(-a2 * x)
3
4 def jacobian_exp(x, a0, a1, a2):
5     df_da0 = x
6     df_da1 = -np.exp(-a2 * x)
7     df_da2 = a1 * x * np.exp(-a2 * x)
8     return np.column_stack((df_da0, df_da1, df_da2))
9
10
11 def modelo_lineal_exponencial(x, a0, a1, a2):
12     return a0 * x - a1 * np.exp(-a2 * x)
13
14 def jacobian_lineal_exp(x, a0, a1, a2):
15     df_da0 = x
16     df_da1 = -np.exp(-a2 * x)
17     df_da2 = a1 * x * np.exp(-a2 * x)
18     return np.column_stack((df_da0, df_da1, df_da2))
19
20
21 def modelo_cuadratico_exponencial(x, a0, a1, a2):
22     return a0 * x**2 - a1 * np.exp(-a2 * x)
23
24 def jacobian_cuadratico_exp(x, a0, a1, a2):
25     df_da0 = x**2
26     df_da1 = -np.exp(-a2 * x)
27     df_da2 = a1 * x * np.exp(-a2 * x)
28     return np.column_stack((df_da0, df_da1, df_da2))
29
30
31 def gauss_newton(x_data, y_data, modelo, jacobiano, a_init, tol=1e-6, max_iter=100):
32     a_current = np.array(a_init, dtype=float) # Inicializaci3n de par metros.
33     iter_count = 0
34
35     while iter_count < max_iter:
36         iter_count += 1
37
38         # Evaluaci3n el modelo en los valores actuales de los par metros.
39         y_pred = modelo(x_data, *a_current)
40         residuals = y_data - y_pred
41
42         # C3lculo del Jacobiano
43         Z = jacobiano(x_data, *a_current)
44
45         #  $(Z^T Z)^{-1} Z^T D$  mediante m3nimos cuadrados.
46         delta_a, _, _, _ = np.linalg.lstsq(Z.T @ Z, Z.T @ residuals, rcond=None)
47         a_current += delta_a
48
49         # Criterio de convergencia.
50         error = np.linalg.norm(delta_a) / np.linalg.norm(a_current)
51         if error < tol:
52             break
53
54     return a_current, iter_count

```